

Amazon API Gateway, carte mentale

PIÈGE classique à l'examen **PRÉCISION** corrige ou complète tes notes Sigles soulignés en pointillé : voir le glossaire en bas

API Gateway

La porte d'entrée de tes backends

01 Les API et la passerelle

UNE API

- **Abstrait l'implémentation** : le consommateur ignore comment le fournisseur travaille
- **N'expose que le nécessaire** : une surface contrôlée, pas tout le système
- Établit un **protocole de communication** entre fournisseur et consommateur
- Flux typique : client → serveur web (API) → base de données, l'API traduit dans les deux sens
- Protocoles possibles : SOAP, WebSockets, HTTP, HTTPS
- Codes de statut : **20x** succès, **40x** erreur client, **50x** erreur serveur

REST ET RESTFUL

- **REST** : un **style d'architecture** (sans état, URI = ressources, verbes HTTP standard)
- **RESTful** : une API qui respecte ces principes. Dans la pratique, les deux mots se confondent

API GATEWAY

- Service **managé et serverless**, la « porte d'entrée » de tes backends
- Fonctionne comme un videur : limite le débit, valide les clés, route selon l'origine, rejette les requêtes invalides avant le backend
- Peut fronter : **Lambda**, **EC2**, tout autre service AWS, des **endpoints VPC**, un système **on-premises** (via Direct Connect)
- Facturation : **par appel reçu** + données sortantes, aucun serveur à gérer

02 Types d'API REST, HTTP ou WebSocket

	REST API	HTTP API
Esprit	Contrôle complet, fonctions avancées	Léger, orienté proxy
Coût	Plus élevé	Latence plus basse, jusqu'à ~70 % moins cher
Exclusif	Mock, transformation fine (VTL), usage plans, API keys, resource policies, cache, génération de SDK client, endpoints edge et privés	Paramètre mapping, auto-déploiement
Choisir si	Fonctions avancées, monétisation, API privée au VPC	Simple passage vers Lambda ou un backend HTTP

Règle : HTTP API par défaut, REST API seulement si tu as besoin d'une de ses fonctions.

WEBSOCKET API

- **Bidirectionnelle** : chaque côté peut envoyer un message à tout moment (pas de requête/réponse)
- Efficace pour les applications « bavardes » en temps réel : chat, tableaux de bord en direct, multijoueur
- **WSS** = la variante sécurisée (TLS) du protocole
- Il faut définir des **route keys** : `$connect`, `$disconnect`, `$default`, plus des routes personnalisées via une route selection expression (ex. `$request.body.action`)

GÉRER L'API

- Développer : console, CLI, ou import/export d'une définition **OpenAPI** (Swagger)
- Déployer : **SAM**, **CloudFormation** ou **CDK**

03 Créer une API REST

URL D'APPEL

```
https://{restapi_id}.execute-api.{region}.amazonaws.com/{stage_name}/
```

- Générée automatiquement, sauf si tu configures un domaine personnalisé

ROUTES ET MÉTHODES

```
GET /products      → liste des produits
GET /products/{id} → un produit
PUT /products/{id} → mise à jour d'un produit
```

- Une route = méthode HTTP + chemin, reliée à une intégration (Lambda, HTTP, service AWS...)

VARIABLE GLOUTONNE

- `{proxy*}` : route fourre-tout (ex. `ANY /{proxy*}`), tout chemin et toute méthode vers **une seule intégration**

- Souvent une Lambda qui fait son propre routage. Évite de déclarer chaque route

MOCK ENDPOINT

- API Gateway renvoie une **réponse prédéfinie**, sans appeler de backend

- Utile pour prototyper un frontend avant que le backend existe

- Sert aussi à gérer les requêtes **preflight CORS** (**OPTIONS**)

IMPORT / EXPORT

- Une API REST ou HTTP se décrit en OpenAPI : import en bloc, ou export pour le versionnement et la portabilité

04 Types d'endpoint

TYPE	POUR	À RETENIR
Edge-optimized	REST seulement	Passer par un point de présence CloudFront automatiquement, protection DDoS intégrée. Pour des clients répartis dans le monde
Regional	REST, HTTP, WebSocket	Servi depuis une seule Région. Bonne pratique : placer ta propre distribution CloudFront devant, pour le CDN et plus de contrôle
Private	REST seulement	Accessible seulement depuis un VPC , via un endpoint VPC

SDK CLIENT

- API Gateway génère des SDK clients pour les API REST : JavaScript, Android, iOS, Ruby, Java

- **PIÈGE** Un SDK client demande en général des **credentials AWS côté client** : risque de sécurité. Des appels HTTPS simples sont préférables, sauf besoin réel du SDK

05 Intégrations ce que l'API appelle derrière elle

LES 5 TYPES

TYPE	PRINCIPE	QUAND
Lambda proxy <code>AWS_PROXY</code>	La requête brute passe telle quelle à Lambda, sa réponse revient telle quelle au client	Cas le plus courant. Lambda gère tout
Lambda non-proxy <code>custom</code>	Échange via des mapping templates VTL qui transforment requête et réponse	Découpler le contrat public de l'API du format interne de la Lambda
Service AWS <code>first-class</code>	Appel direct : <code>SQS SendMessage</code> , <code>Step Functions StartExecution</code> ...	Pas de Lambda entre les deux : une couche de calcul en moins
HTTP proxy	Transfère vers un autre endpoint HTTP(S) (EC2, API externe)	Migrer un monolithe vers des microservices, route par route
VPC Link	Atteint des ressources d'un VPC sans endpoint public (ex. ALB interne, NLB)	Backend privé, non exposé à Internet

LAMBDA PROXY : LA FORME OBLIGATOIRE

- **PIÈGE** La Lambda **doit** renvoyer exactement cette forme, avec `body` en **chaîne**. Sinon API Gateway renvoie une erreur 502

```
{
  "statusCode": 200,
  "headers": { "Content-Type": "application/json" },
  "body": "{ \"message\": \"ok\" }",
  "isBase64Encoded": false
}
```

- Exemple d'erreur 403 : `statusCode: 403, body: \"{ \"error\": \"Forbidden\" }\"`

- Avec le proxy, les en-têtes CORS sont aussi à renvoyer par la Lambda (voir la carte DynamoDB, branche 12)

LE TRAJET D'UNE REQUÊTE

aller Method Request → Integration Request

retour Integration Response → Method Response

- Le client entre par **Method Request**, le backend est appelé après **Integration Request**, et la réponse repart dans l'ordre inverse

- Chaque étape est un point où API Gateway peut **valider**, **transformer** ou **rejeter**

PARAMETER MAPPING (HTTP API)

- Alternative plus légère aux templates VTL des API REST
- Ajoute, écrase ou retire des en-têtes, query strings ou paramètres de chemin au passage
- Exemple : injecter `$context.requestId` comme en-tête avant le backend

06 Déploiement : stages, variables, canary

STAGES

- Un **stage** est une version nommée de l'API : `dev`, `beta`, `prod`. Son nom apparaît dans l'URL d'appel
- **Obligatoire** pour REST et WebSocket : sans déploiement sur un stage, l'API n'est pas joignable
- **Automatique** pour HTTP : stage par défaut, avec option d'auto-déploiement à chaque changement

- Un stage peut représenter un environnement, chacun avec un backend différent

- **PRÉCISION** Sur une API REST, une modification n'est visible qu'après un **nouveau déploiement** sur le stage

VARIABLES DE STAGE

- Paire clé/valeur propre à un stage, utilisable dans la config d'une intégration

- Exemple : `${stageVariables.lambdaAlias}` vaut `dev`, `beta` ou `prod` selon le stage

- La **même définition d'API** sert des backends différents, sans la dupliquer

DÉPLOIEMENT CANARY

- Envoie un petit pourcentage du trafic de **production** vers la nouvelle version, le reste va à la version stable
- Config : `percentTraffic` (ex. 10,5 %) et, au besoin, `stageVariableOverrides`
- Surveiller dans CloudWatch, augmenter petit à petit
- **Promouvoir** le canary = il devient la version à 100 % du trafic, et sa config est retirée

07 Contrôle d'accès se protéger du trafic indésirable et autoriser les bons appelants

SE PROTÉGER DU TRAFIC INDÉSIRABLE

MÉCANISME	CE QU'IL FAIT
Resource policy	Limite qui peut appeler l'endpoint : un compte AWS précis, ou une plage d'IP (ex. celle du bureau). API REST seulement
Domaine perso + certificats client	Domaine avec certificat SSL (ACM), puis certificats clients (valides 365 jours) : le backend vérifie qu'un appel vient bien d'API Gateway, ou d'un appareil IoT de confiance
AWS WAF	Pare-feu applicatif devant API Gateway. Bloque injections SQL et XSS, filtre par motif (en-têtes, corps, URI), par plage d'IP, pays ou bots. Règles gérées par AWS
CORS	Règle du navigateur pour les requêtes AJAX cross-origine (domaine, sous-domaine, port ou protocole différent). Les requêtes « simples » (GET) sont exemptées
Throttling	Protège contre la surcharge. Requête limitée = HTTP 429 « Too Many Requests »

- **PRÉCISION** Deux choses distinctes : les **certificats client** générés par API Gateway (le backend vérifie l'appelant), et le **mutual TLS** sur un domaine personnalisé (les clients présentent un certificat validé contre un truststore)

CORS EN DÉTAIL

- Requête non simple → le navigateur envoie d'abord un **preflight OPTIONS** que le serveur doit approuver (origines, méthodes, en-têtes autorisés)
- API Gateway peut gérer le preflight seul (un mock en coulisse pour REST, plus simple pour HTTP), ce qui décharge ton code
- **PIÈGE** Avec une intégration Lambda proxy, le preflight est géré, mais la vraie réponse doit porter elle-même les en-têtes CORS

THROTTLING ET USAGE PLANS

- Quota Région par défaut : **10 000** requêtes/seconde pour toutes les API du compte (relevable sur demande)

- **Réglable par stage**, par **méthode / route**, et par **client** via un usage plan

- Ordre d'application, du plus précis au plus général : client + méthode → client → méthode / stage → compte

- **Usage plan** (REST) : quels clients (identifiés par une **API key**) accèdent à quels stages, avec une limite de débit et un **quota** (par jour, semaine, mois). Plusieurs paliers possibles (User, Partner, Service)

- **PIÈGE** Une API key identifie et mesure un client, elle ne remplace pas une vraie authentification

AUTORISER LES APPELANTS : 4 OPTIONS

OPTION	PRINCIPE	QUAND
None	Accès ouvert à tous	API publique
IAM	L'appelant signe la requête avec des credentials AWS (souvent temporaires, via un SDK). Une requête non signée est rejetée	Appellants AWS, services internes
JWT	Jeton validé derrière OIDC / OAuth 2.0. REST : Cognito user pool. HTTP : Cognito ou un autre fournisseur JWT. Vérifie le jeton et son scope	Utilisateurs d'une application web ou mobile
Lambda authorizer	Une Lambda s'exécute avant l'intégration et valide un jeton bearer (OAuth maison, SAML) ou d'autres paramètres avec ta logique	Règles d'accès sur mesure

- Un authorizer se configure par route, ou se partage entre routes

- **PRÉCISION** Lambda authorizer : c'est l'**authorizer** qui renvoie le verdict (une policy IAM, ou `isAuthorized` true/false pour une HTTP API). La fonction cible reçoit ensuite les données `context` de l'authorizer, ce qui lui permet de gérer un refus proprement

08 Surveillance

MÉTRIQUES CLOUDWATCH (AUTOMATIQUES, CHAQUE MINUTE)

MÉTRIQUE	SENS
<code>Count</code>	Nombre total d'appels sur la période
<code>IntegrationLatency</code>	Temps de réponse du backend seul
<code>Latency</code>	Aller-retour complet : de la réception de la requête par API Gateway à la réponse au client

`4XXError` / `5XXError` Erreurs côté client / côté serveur

- Aucune config requise, données **côtées 15 mois**

- `Latency` moins `IntegrationLatency` = le temps passé dans API Gateway lui-même

ACCESS LOGS (OPTIONNELS, PAR STAGE)

- 1. Créer un **log group** CloudWatch

- 2. Le pointer dans les paramètres d'accès log du stage, avec un format (Common Log Format ou JSON) : IP source, heure, méthode, route, statut, taille de réponse, ID de requête

- Les logs d'**erreur** sont créés automatiquement. Les access logs demandent cette config explicite

LES 3 AUTRES OUTILS

OUTIL	RÉPOND À
X-Ray	« Où une transaction passe-t-elle son temps dans mon application distribuée ? » Regroupe les données de chaque service touché en une trace , avec la distribution des latences. Activable directement sur API Gateway
Config	« Cette ressource respecte-t-elle nos règles ? » Snapshot de configuration, règles de conformité, notification SNS en cas d'écart
CloudTrail	« Qui a modifié quoi, et quand ? » Activé par défaut, enregistre chaque appel d'API lié à API Gateway (console, SDK, CLI)

09 Optimisation

CACHE (API REST)

- Met en cache les réponses aux **GET** : moins d'appels au backend, latence réduite
- Clés de cache possibles : chemin, en-têtes, query strings
- Réglable **par stage** ou **par méthode**, avec un **TTL** modifiable. Chiffrement du cache possible
- **PIÈGE** Facturé à **l'heure, utilisé ou non** : un coût fixe, pas du paiement à l'usage
- Métriques : `CacheHitCount` (servi par le cache) et `CacheMissCount` (servi par le backend)

COMPRESSION DES PAYLOADS

- Réduit les données échangées client ↔ API Gateway : moins de coût et de latence

- Codages : `deflate`, `gzip`, `identity`

- La `minimumCompressionSize` fixe le seuil ; 0 = pas de compression

- Le **décompression** des requêtes est activé par défaut, la **compression** des réponses se configure

- **PIÈGE** Compresser un tout petit payload peut l'agrandir, et coûte du calcul des deux côtés : tester le bon seuil

10 Scénario du lab : le café

- Application web (hébergée sur S3) qui lit et écrit la table DynamoDB `FoodProducts` **via une API**, pas directement depuis le JavaScript du navigateur

- Démarche : 1. API avec des **mock endpoints** → 2. validation par les parties prenantes → 3. remplacement des mocks par de vraies intégrations (Lambda ↔ DynamoDB)

- Le contrat public de l'API ne change pas : API Gateway reste une porte d'entrée stable que l'on peut pointer vers d'autres backends

- Points à ne pas oublier avec ce montage : les en-têtes CORS dans la réponse de la Lambda, et la conversion des `Decimal` avant `json.dumps` (voir la carte DynamoDB)

11 Hors de tes notes à connaître pour le DVA-C02

LES CODES D'ERREUR À RECONNAÎTRE

CODE	CAUSE TYPIQUE
400	Requête invalide, ou refusée par la validation de requête
401	Non autorisé : jeton absent ou invalide pour l'authorizer
403	Refusé : IAM, resource policy, WAF, API key manquante. « Missing Authentication Token » = souvent une route ou un chemin inexistant
429	Throttling : trop de requêtes
502	Réponse du backend invalide, typiquement une Lambda proxy qui ne renvoie pas la forme attendue
504	Timeout d'intégration

LIMITES

- **PIÈGE** Timeout d'intégration : **29 s** par défaut pour une API REST (30 s pour une HTTP API)

- Depuis juin 2024, relevable au-delà de 29 s pour les REST régionales et privées, avec un impact possible sur le quota de débit

- Payload max : **10 MB** par requête ou réponse. Une Lambda appelée en synchrone : 6 MB

- Throttling : **10 000** req/s en régime établi, avec un **burst** de **5 000** requêtes

- Message WebSocket : **128 KB**

AUTHORIZER ET CACHE

- Le résultat d'un Lambda authorizer est mis en cache : TTL par défaut **300 s** (0 à 3 600 s)

- Cache des réponses : TTL par défaut **300 s**, maximum **3 600 s**

- Invalider le cache : l'appelant envoie l'en-tête `Cache-Control: max-age=0`, avec la permission d'invalider

VALIDATION ET DOMAINE PERSO

- **Validation de requête** (REST) : paramètres, en-têtes et corps vérifiés contre un **modèle JSON Schema** avant d'appeler le backend → **400** sinon. Économise des invocations Lambda

- Domaine personnalisé : certificat **ACM**. Endpoint edge-optimized : certificat en `us-east-1`. Endpoint régional : certificat dans la même Région

WEBSOCKET, SAM, VARIABLES

- Pour écrire à un client connecté derrière le backend : appeler l'API `@connections` avec son `connectionId`

- SAM : un event `Api` crée une API REST, un event `HttpApi` crée une API HTTP. `sam local start-api` les simule en local

- Variable de stage qui désigne une Lambda : API Gateway doit avoir la permission de l'invoquer (`lambda:InvokeFunction`), à ajouter à la main

12 Glossaire des sigles

API	Application Programming Interface	REST	Representational State Transfer	WSS	WebSocket Secure (WebSocket sur TLS)
VTL	Velocity Template Language (mapping templates)	OIDC	OpenID Connect	JWT	JSON Web Token
WAF	Web Application Firewall	mTLS	Mutual TLS (client et serveur se présentent un certificat)	ACM	AWS Certificate Manager
IAM	Identity and Access Management	VPC	Virtual Private Cloud	ALB	Application Load Balancer
NLB	Network Load Balancer	CORS	Cross-Origin Resource Sharing	TTL	Time To Live (durée de vie dans le cache)
SAM	Serverless Application Model	CDK	Cloud Development Kit	SDK	Software Development Kit
SNS	Simple Notification Service	SQS	Simple Queue Service	DDoS	Distributed Denial of Service
CDN	Content Delivery Network (CloudFront)				

Les réflexes pour l'examen

- Simple passage vers Lambda, au moindre coût → HTTP API. Besoin d'API keys, usage plans, cache, mock ou API privée → REST API.
- Échange bidirectionnel en temps réel (chat, direct) → WebSocket API avec ses route keys.
- Clients répartis dans le monde → edge-optimized. Accessible seulement depuis un VPC → private. Les deux sont REST seulement.
- Lambda proxy → renvoyer `statusCode`, `headers`, `body` (chaîne). Sinon erreur 502.
- Transformer requête et réponse sans toucher la Lambda → intégration non-proxy avec templates VTL.
- Appeler SQS ou Step Functions sans Lambda → intégration directe vers le service AWS.
- Lambda un monolithe petit à petit → intégration HTTP proxy, route par route.
- Atteindre un ALB ou NLB privé dans un VPC → VPC Link.
- Une API, plusieurs environnements → stages et variables de stage (alias Lambda).
- Tester une nouvelle version sur un peu de trafic réel → déploiement canary (`percentTraffic`).
- Changement d'une API REST invisible → redéploier sur le stage.
- Erreur 429 → throttling. Limiter par client → usage plan avec API key et quota.
- Restreindre par IP ou par compte → resource policy (REST seulement). Bloquer injections SQL, XSS, pays → WAF.
- Le backend doit vérifier que l'appel vient d'API Gateway → certificat client. Authentifier le client lui-même → mTLS sur domaine personnalisé.
- Utilisateurs d'une app → Cognito / JWT. Logique de jeton sur mesure → Lambda authorizer. Appellants AWS → IAM.
- Une API key n'est pas un mécanisme de sécurité.
- Erreur CORS dans le navigateur → activer CORS (preflight OPTIONS). Avec Lambda proxy, la Lambda renvoie elle-même les en-têtes.
- 502 → réponse Lambda mal formée. 504 → timeout (29 s). 403 « Missing Authentication Token » → mauvaise route.
- Latency contre IntegrationLatency pour situer la lenteur. Suivre une requête à travers les services → X-Ray. Qui a modifié la config → CloudTrail. Conformité → Config.
- Réduire les appels backend sur les GET → cache (facturé à l'heure). Réduire la taille des échanges → compression.